

Cursor Bash + Git + GitHub

從零初始化、推送遠端與中文 Commit 工作流

專案：management-review-agent | 使用環境：Windows + Cursor + Git Bash

目前狀態：Git 已安裝，Git 使用者名稱與 email 已設定；management-review-agent 尚未初始化為 Git repository。本手冊只使用 Git 與瀏覽器，不使用 GitHub CLI。

這份手冊的目標

建立一個乾淨的 Git repository，把 Day 1、Day 2 的程式與規格文件推到自己的 GitHub repository，並從此使用小而清楚、以中文摘要為主的 commit 與分支保留學習歷程。

1. 先理解 Git、GitHub 與 remote

名詞	意思
工作目錄	你在 Cursor 編輯的檔案，例如 src/analysis.py。
Git repository	本機的版本歷史；由 .git 目錄保存。
commit	一個有訊息、可回看的版本快照。
branch	從同一份歷史延伸出的獨立工作線。
remote / origin	本機 Git 對應的遠端 GitHub repository。origin 是慣例名稱。
push	把本機 commit 上傳到遠端。
pull	取得遠端 commit 並整合到本機。
pull request	在 GitHub 上請求把分支併回 main 的審查紀錄。

重要：Git 不會自動備份所有檔案。只有經過 git add，再 git commit，最後 git push 的內容，才會出現在 GitHub。

2. 不應提交的檔案

虛擬環境可由 requirements 重新建立，不能放入 Git。密碼、token、真實資料也不可提交。請先在 Cursor 專案根目錄建立 .gitignore。

```
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.coverage
.env
.env.*
!.env.example
.DS_Store
```

保留並提交的內容通常包括：src/、tests/、sql/、knowledge/、skills/、文件、Docker Compose、requirements.txt、README.md 與虛構 raw data。

3. 初始化本機 Git repository

```
cd "/c/Users/huang/Desktop/健鼎面試/management-review-agent"
```

```
# 在目前專案建立 Git repository，主分支直接命名為 main  
git init -b main
```

```
# 確認 Git 看見哪些檔案  
git status
```

```
# 將要納入第一版的檔案加入暫存區  
git add .
```

```
# 最後一次確認：venv、.env 不應出現在這裡  
git status
```

```
# 建立第一個本機版本  
git commit -m "chore: 初始化 management review agent 專案"
```

若 git status 出現 venv/、.env 或任何真實敏感資料，先修正 .gitignore，再執行 git add .。不要直接 commit。

4. 建立自己的 GitHub repository

本手冊只使用 GitHub 網站建立遠端 repository；之後所有版本操作都使用 Cursor Bash 的 Git CLI。

1. 登入 github.com。
2. 點右上角加號，選 New repository。
3. Repository name 填 management-review-agent。
4. 選 Private 或 Public。練習與求職展示通常可選 Public；若含未公開內容則選 Private。
5. 不要勾選 README、.gitignore 或 License，因為本機已有專案。
6. 按 Create repository。
7. 複製 HTTPS URL。

把 YOUR_USERNAME 換成你的 GitHub 帳號

```
git remote add origin https://github.com/YOUR_USERNAME/management-review-agent.git
```

```
git push -u origin main
```

確認遠端是否正確

```
git remote -v
```

```
git branch -vv
```

```
git log --oneline --decorate -5
```

不要把 GitHub token 寫進 remote URL、程式碼、.env.example 或 commit message。第一次 git push 若要求驗證，依 Git Credential Manager 開啟的瀏覽器流程登入 GitHub。

5. 每日工作流程

main 保持可運作。每一個明確小功能使用一個 branch，例如 Day 2 的 SQL 分析。

```
# 先回到 main 並取得遠端最新版本
git switch main
git pull --ff-only origin main
```

```
# 建立功能分支
git switch -c feat/day2-sql-analysis
```

```
# 編輯檔案後，先看改了什麼
git status
git diff
```

```
# 只加入本次工作相關檔案
git add src/analysis.py sql/kpi_queries.sql tests/test_analysis.py
```

```
# 檢查即將提交的内容
git diff --staged
```

```
# 建立清楚的 commit
git commit -m "feat(analysis): 新增 KPI SQL 分析"
```

```
# 第一次 push 這個新分支
git push -u origin feat/day2-sql-analysis
```

Commit message 建議採 Conventional Commits 格式：type(scope): 中文摘要。type 保留英文慣例，摘要使用中文，兼顧團隊辨識與你的閱讀習慣。

Commit 類型	中文範例
feat	feat(analysis): 新增預算差異查詢
fix	fix(analysis): 修正離職率零分母處理
test	test: 新增重複 KPI 主鍵測試
docs	docs: 補充材料耗損率定義
chore	chore: 初始化專案 repository

6. GitHub Pull Request 與併回 main

push 分支後，GitHub 網頁通常會出現 Compare & pull request。填寫這次改了什麼、如何測試、已知限制。完成後 merge 到 main。

```
# PR 合併後，回到本機同步 main
git switch main
git pull --ff-only origin main
```

```
# 確認分支已不需要時才刪除本機分支
git branch -d feat/day2-sql-analysis
```

建立 Pull Request 時，從 GitHub 網頁選擇 Compare & pull request，確認 base 是 main、compare 是你的功能分支，再填寫改動內容、測試方式與已知限制。

7. 最常用的檢查與安全復原

目的	指令	注意事項
看目前狀態	git status	每天開始與 commit 前都應執行。
看尚未暫存的變更	git diff	不會修改檔案。
看即將 commit 的變更	git diff --staged	避免把錯誤檔案送進歷史。
取消暫存但保留修改	git restore --staged FILE	安全，工作內容仍留在磁碟。
丟棄未暫存修改	git restore FILE	會覆寫工作內容，確認不需要後才用。
查看歷史	git log --oneline --decorate	不會修改任何內容。

不要使用：git reset --hard，除非你完全理解它會丟棄哪些尚未提交的修改。初學階段先用 git status、git diff 與 git restore --staged，就足夠處理多數情況。

8. 你的首次 push 檢查表

- [] 我位於 management-review-agent 專案根目錄，而不是外層的健鼎面試資料夾。
- [] .gitignore 已排除 venv、.env、__pycache__ 與測試暫存檔。
- [] git status 沒有顯示不該提交的敏感資料或巨大檔案。
- [] git commit 已成功建立。
- [] GitHub repo 是我自己的，且可見性選擇正確。
- [] git remote -v 的 origin URL 指向我的 GitHub repository。
- [] git push -u origin main 成功。
- [] GitHub 網頁能看到 README、src、data、knowledge、skills 等檔案。

建議第一個 commit：先提交 Day 1 與目前 Day 2 的資料、文件與 analysis.py。接下來 SQL、測試、報告各自用獨立 commit，這會讓你面試時能清楚說明專案如何逐步演進。

9. 分支操作：建立、切換、查看與刪除

分支是把一個工作主題隔開的方式。它不是複製整個專案；Git 只記錄歷史上的分岔點，所以成本很低。建議一個 branch 只處理一個可描述的任務。

情境	指令	說明
查看本機分支	git branch	目前所在分支前面會有 *。
查看本機與遠端分支	git branch -a	-a = all，列出 local 與 remote。
建立並切換分支	git switch -c feat/day2-sql-analysis	-c = create。新名稱採 type/主題。
切換既有分支	git switch main	切換前先 git status；未提交修改可能阻擋切換。
從遠端建立本機追蹤分支	git switch -c fix/kpi-zero --track origin/fix/kpi-zero	把遠端分支下載成可工作的本機分支。
安全刪除已合併分支	git branch -d feat/day2-sql-analysis	-d = delete；未合併時 Git 會阻止刪除。
強制刪除分支	git branch -D experiment/chart	-D = force delete；確認不需要該分支 commit 才能用。

命名例：feat/day2-sql-analysis、fix/zero-denominator、docs/kpi-dictionary、test/anomaly-rules。不要使用「test」、「new」、「backup」這種看不出目的的名稱。

10. 同步遠端：fetch、pull、push

指令	實際做什麼	適合何時用
git fetch origin	只下載遠端提交與分支資訊，不改目前工作檔案。	想先確認遠端變了什麼。
git pull --ff-only origin main	fetch 後，只有能快轉時才整合 main。	每天開始工作前；安全且歷史清楚。
git pull	下載並嘗試整合遠端變更；可能產生 merge commit。	理解其效果後才使用。
git push	上傳目前分支的新 commit 到其追蹤遠端。	已設定 upstream 後。
git push -u origin feat/day2-sql-analysis	首次上傳分支，同時設定追蹤關係。	新分支第一次 push。
git push origin --delete feat/day2-sql-analysis	刪除 GitHub 上的遠端分支。	PR 已合併且團隊確認不再使用。

推薦同步流程：git status → git switch main → git pull --ff-only origin main → git switch -c 新分支。若你有未提交修改，先 commit 或先用 git stash，避免切換時把工作混在一起。

若 push 被拒絕，通常表示遠端已有你尚未取得的 commit。先確認 git status 乾淨，再 git pull --ff-only origin 分支名；若無法快轉，不要急著強推，先依「衝突處理」章節檢查原因。

11. Merge：把工作成果整合回 main

最適合目前學習流程的方式是：功能分支 push 後，在 GitHub 網站開 Pull Request，確認內容後由網站合併。這會保留討論、檢查與變更紀錄。

GitHub 網站 Merge 的建議

1. 在功能分支頁面按 Compare & pull request。
2. base 選 main；compare 選你的功能分支。
3. 寫清楚「改了什麼、如何測試、是否有已知限制」。
4. 檢查 Files changed，確認沒有 venv、.env 或無關檔案。
5. 選 Create pull request，確認後按 Merge pull request。
6. 回到本機執行下方同步指令。

```
git switch main
git pull --ff-only origin main
git branch -d feat/day2-sql-analysis
# GitHub 網站已刪除遠端 branch 時，可清除本機過期參照
git fetch --prune origin
```

也可以只在本機 merge，但多人協作或需要留下學習紀錄時，優先用 PR：

```
git switch main
git pull --ff-only origin main
git merge --no-ff feat/day2-sql-analysis
git push origin main
```

--no-ff：即使 Git 可以直接快轉，也建立一個 merge commit；優點是歷史中看得出這是一個完整功能分支。個人小專案可用 GitHub 預設合併方式，不必每次手動指定。

12. 衝突處理：兩人改到同一段程式時

衝突不是程式壞掉，而是 Git 無法判斷要保留哪一份修改。不要直接刪除標記；先理解兩邊要達到的結果。

git status

開啟衝突檔案，會看到：

<<<<<< HEAD

本機目前分支的內容

=====

要合併進來的內容

>>>>>> origin/main

手動保留正確內容後，移除所有衝突標記

git add src/analysis.py

git commit

完成後再 git push

情境	暫停並回到合併前
git merge 時發生衝突	git merge --abort
git pull 造成 merge 衝突	git merge --abort (若 status 顯示正在 merge)
rebase 時發生衝突	git rebase --abort

完成衝突處理後必做：執行測試，例如 `.\venv\Scripts\python.exe -m pytest`；再用 `git diff --staged` 確認你確實保留了正確邏輯。衝突解完能 commit，不代表程式語意一定正確。

13. 暫存工作：git stash

stash 是暫時把尚未 commit

的修改收起來，讓工作目錄回到乾淨狀態。適合突然需要切到另一個分支修小問題；不適合拿來長期保存進度。

```
# 暫存目前已追蹤檔案的修改，並寫清楚原因
git stash push -m "WIP: Day 2 異常規則草稿"
```

```
# 若也要包含新建立但尚未 git add 的檔案
git stash push -u -m "WIP: SQL 草稿"
```

```
# 查看 stash 清單
git stash list
```

```
# 套回最新一筆 stash，並從清單移除
git stash pop
```

```
# 套回但保留 stash，確認無誤後再手動刪除
git stash apply
git stash drop
```

-u：--include-untracked，連尚未被 Git 追蹤的新檔案一起暫存。它不包含被 .gitignore 忽略的檔案。要暫存 ignored 檔案的 -a 風險較高，初學者通常不需要。

14. 修改、暫存與復原情境

你遇到的情況	推薦做法	結果
改錯檔案，尚未 git add	git restore src/analysis.py	丟棄該檔案未暫存修改。先 git diff 確認。
git add 錯檔，但想保留檔案內容	git restore --staged FILE	只取消暫存，工作檔內容保留。
commit 訊息打錯，尚未 push	git commit --amend -m "docs: 正確摘要"	改寫最後一個 commit。
漏加一個檔案，尚未 push	git add FILE ; git commit --amend --no-edit	把檔案補進最後 commit。
已 commit、想安全撤回效果	git revert COMMIT_HASH	建立一個反向 commit，不改寫公開歷史。
想查看某版本內容	git show COMMIT_HASH	只讀檢查該 commit。

已 push、他人可能已拉取的 commit，不要隨意 amend、rebase 或 force push。要撤銷功能時以 git revert 為優先，因為歷史可追溯。

COMMIT_HASH：用 git log --oneline 看到前面那串短代號，例如 a1b2c3d。執行 git revert a1b2c3d 前，先用 git show a1b2c3d 確認它真的是要撤的版本。

15. 常用短選項速查表

選項	完整名稱或意義	常見用途
-v	verbose，顯示更詳細資訊	git commit -v：在提交編輯器中附帶 diff，方便檢查。
-u	依指令不同而不同	git push -u：設定 upstream；git stash -u：包含未追蹤檔案。
-m	message	git commit -m "docs: 更新手冊"；git stash -m "WIP"。
-c	create	git switch -c 新分支：建立並切換。
-a	all（依指令不同）	git branch -a：列出全部分支。不是 git add 的替代品。
-d / -D	delete / force delete	git branch -d 安全刪分支；-D 強制刪。
-n	dry run 或 no edit（視指令而定）	先讀 git 指令說明，避免猜測；短選項會因指令改變意義。

Git 的短選項不是全域一致的。例如 -u 在 git push 是 upstream，在 git stash 是 include-untracked；所以遇到不確定的選項，請查當下指令說明：

```
git push -h
git stash -h
git help commit
# 在 Bash 內按 q 可離開較長的說明畫面
```

最值得養成的習慣不是背所有旗標，而是在任何有疑慮的操作前跑 git status、git diff、git diff --staged。這三個指令幾乎不會傷害資料。

16. 常見開發情境一覽

情境	你可以照做的最短流程
開始一個新功能	switch main → pull --ff-only → switch -c feat/... → 開發 → add → commit → push -u。
想確認別人是否更新 main	switch main → fetch origin → log main..origin/main --oneline。
PR 合併後清理	switch main → pull --ff-only → branch -d 功能分支 → fetch --prune origin。
工作做到一半要修緊急問題	stash push -m WIP → switch main → pull → switch -c fix/... → 修正並 push。
發現 commit 漏了檔案，尚未 push	add 漏檔 → commit --amend --no-edit → push。
發現已 push 的功能不該存在	show HASH → revert HASH → 測試 → push。
pull 或 merge 出現衝突	status → 看標記 → 手動整合 → add → commit → 測試 → push。

Day 2 的建議實作分支順序

feat/day2-sql-analysis # SQL 與 Python 計算比對
test/day2-data-quality # 資料品質測試
feat/day2-incident-trace # 事件關聯與證據欄位
docs/day2-analysis-notes # 補充方法與限制

每完成一個小主題就 commit。這比等到 Day 2 全做完才產生一個巨大 commit 更容易檢討、回看，也更能呈現你的工程習慣。